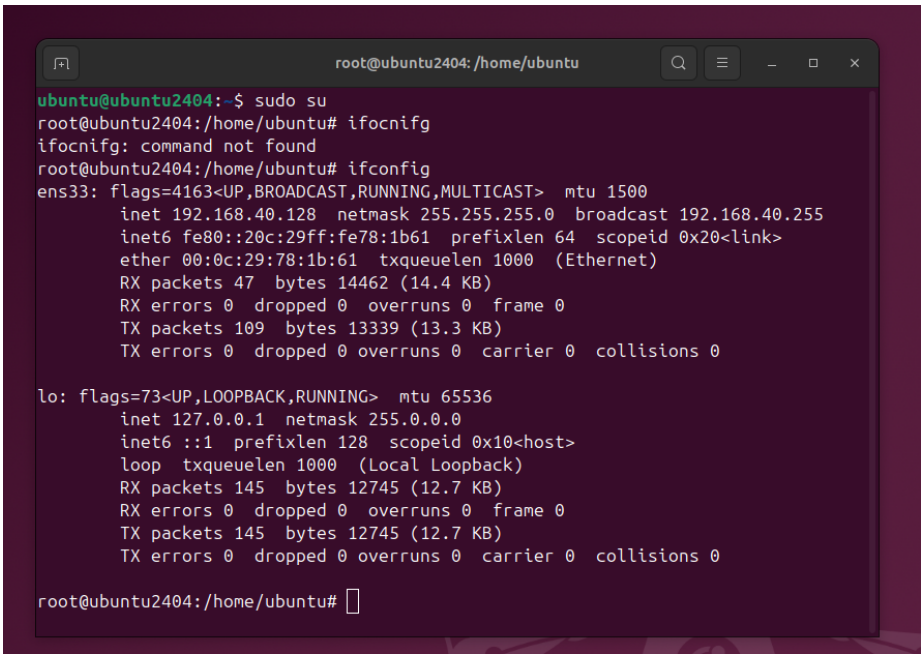
IT203 – Lab 09 Firewall & Intrusion Detection System (IDS)

Nicholas Kehayias & Dustin Crawford

Introduction:

In this lab we will be learning how intrusion detection systems work. We will be doing this by using a Kali Linux system to target an Ubuntu Linux system. This sets up a virtual environment to simulate different intrusion systems without creating any real damage.

Exercise 1:



Running ifconfig on ubuntu to get IP address

```
root@ubuntu2404: /etc/snort
  GNU nano 7.2                    snort.conf *
include $RULE_PATH/exploit.rules
#include $RULE_PATH/file-executable.rules
#include $RULE_PATH/file-flash.rules
#include $RULE_PATH/file-identify.rules
#include $RULE_PATH/file-image.rules
#include $RULE_PATH/file-multimedia.rules
#include $RULE_PATH/file-office.rules
#include $RULE_PATH/file-other.rules
#include $RULE_PATH/file-pdf.rules
include $RULE_PATH/finger.rules
include $RULE_PATH/ftp.rules
# ICMP standard information queries will trigger these rules, they are very
# chatty, only enable if you need them
include $RULE_PATH/icmp-info.rules
include $RULE_PATH/icmp.rules
include $RULE_PATH/imap.rules
#include $RULE_PATH/indicator-compromise.rules
#include $RULE_PATH/indicator-obfuscation.rules
#include $RULE_PATH/indicator-shellcode.rules
include $RULE_PATH/info.rules

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

Configuring Snort conf on Ubuntu

```
root@ubuntu2404: /etc/snort
          Using ZLIB version: 1.3

          Rules Engine: SF_SNORT_DETECTION_ENGINE  Version 3.2  <Build 1>
          Preprocessor Object: SF_FTPTELNET  Version 1.2  <Build 13>
          Preprocessor Object: SF_SIP  Version 1.1  <Build 1>
          Preprocessor Object: SF_POP  Version 1.0  <Build 1>
          Preprocessor Object: SF_IMAP  Version 1.0  <Build 1>
          Preprocessor Object: SF_REPUTATION  Version 1.1  <Build 1>
          Preprocessor Object: SF_SDF  Version 1.1  <Build 1>
          Preprocessor Object: SF_DNP3  Version 1.1  <Build 1>
          Preprocessor Object: SF_DNS  Version 1.1  <Build 4>
          Preprocessor Object: SF_SSH  Version 1.1  <Build 3>
          Preprocessor Object: SF_SMTP  Version 1.1  <Build 9>
          Preprocessor Object: SF_SSLPP  Version 1.1  <Build 4>
          Preprocessor Object: SF_MODBUS  Version 1.1  <Build 1>
          Preprocessor Object: appid  Version 1.1  <Build 5>
          Preprocessor Object: SF_S7COMMPLUS  Version 1.0  <Build 1>
          Preprocessor Object: SF_DCERPC2  Version 1.0  <Build 3>
          Preprocessor Object: SF_GTP  Version 1.1  <Build 1>

Total snort Fixed Memory Cost - MaxRss:106132
Snort successfully validated the configuration!
Snort exiting
root@ubuntu2404:/etc/snort#
```

Validation of snort config file

Questions:

1. **What is the snort.conf file used for exactly and where does it get its set of rules from?**
   The snort.conf file is the primary file for Snort intrusion detection. That file gets its rules from external files and sources.
2. **Pick five rules from the set of rules in the snort.conf file and explain them:**
   1) **Detecting a simple ICMP ping to a specific external server.**
      This rule tells Snort to generate an alert anytime any computer attempts to send an ICMP (ping) packet toward the specific IP address 8.8.8.8, logging a message about the activity.
   2) **Alerting on traffic to the insecure Telnet port**
      This rule alerts administrators if anyone attempts to connect using the insecure Telnet protocol (port 23) to any internal machine within the defined $HOME_NET.
   3) **Detecting a potential SQL Injection attempt in HTTP traffic**
      Snort will alert if traffic coming from outside ($EXTERNAL_NET) directed at internal web servers ($HTTP_PORTS) contains the text "SELECT" within the data payload, which could indicate a potential SQL injection attack.
   4) **Logging all web server traffic for later analysis.**
      Instead of alerting, this rule simply records a log of all established internal web server traffic flowing out to external destinations for later review or forensic analysis.
   5) **Blocking a known "evilbot" user-agent in HTTP headers.**
      This rule is proactive: if a network connection tries to reach an internal web server and identifies itself with the exact user-agent string "evilbot" in the HTTP header, Snort will immediately block (drop) the connection.

Key Takeaways:

- How to install and configure Snort within Ubuntu

Exercise 2

```
┌──(root㉿kali)-[/home/kali]
└─# ping 192.168.40.128
PING 192.168.40.128 (192.168.40.128) 56(84) bytes of data.
64 bytes from 192.168.40.128: icmp_seq=1 ttl=64 time=1.16 ms
64 bytes from 192.168.40.128: icmp_seq=2 ttl=64 time=0.735 ms
64 bytes from 192.168.40.128: icmp_seq=3 ttl=64 time=0.668 ms
64 bytes from 192.168.40.128: icmp_seq=4 ttl=64 time=0.398 ms
64 bytes from 192.168.40.128: icmp_seq=5 ttl=64 time=0.686 ms
64 bytes from 192.168.40.128: icmp_seq=6 ttl=64 time=0.646 ms
64 bytes from 192.168.40.128: icmp_seq=7 ttl=64 time=0.703 ms
64 bytes from 192.168.40.128: icmp_seq=8 ttl=64 time=0.666 ms
64 bytes from 192.168.40.128: icmp_seq=9 ttl=64 time=0.681 ms
64 bytes from 192.168.40.128: icmp_seq=10 ttl=64 time=0.667 ms
64 bytes from 192.168.40.128: icmp_seq=11 ttl=64 time=0.597 ms
64 bytes from 192.168.40.128: icmp_seq=12 ttl=64 time=0.582 ms
64 bytes from 192.168.40.128: icmp_seq=13 ttl=64 time=1.09 ms
64 bytes from 192.168.40.128: icmp_seq=14 ttl=64 time=0.655 ms
64 bytes from 192.168.40.128: icmp_seq=15 ttl=64 time=0.839 ms
64 bytes from 192.168.40.128: icmp_seq=16 ttl=64 time=0.648 ms
64 bytes from 192.168.40.128: icmp_seq=17 ttl=64 time=0.838 ms
64 bytes from 192.168.40.128: icmp_seq=18 ttl=64 time=0.742 ms
64 bytes from 192.168.40.128: icmp_seq=19 ttl=64 time=0.773 ms
64 bytes from 192.168.40.128: icmp_seq=20 ttl=64 time=0.704 ms
64 bytes from 192.168.40.128: icmp_seq=21 ttl=64 time=0.440 ms
64 bytes from 192.168.40.128: icmp_seq=22 ttl=64 time=47.8 ms
64 bytes from 192.168.40.128: icmp_seq=23 ttl=64 time=0.650 ms
64 bytes from 192.168.40.128: icmp_seq=24 ttl=64 time=0.380 ms
64 bytes from 192.168.40.128: icmp_seq=25 ttl=64 time=0.586 ms
^C
--- 192.168.40.128 ping statistics ---
25 packets transmitted, 25 received, 0% packet loss, time 24372m
s
rtt min/avg/max/mdev = 0.380/2.574/47.813/9.235 ms
```

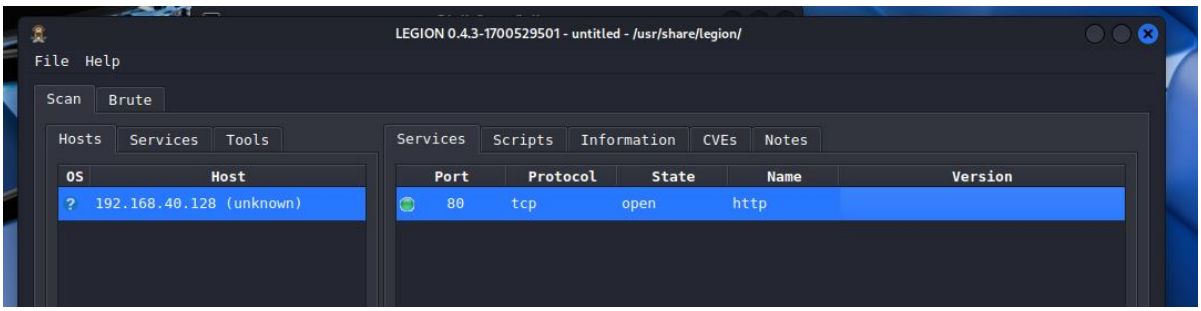
Pinging ubuntu machine from Kali

```
activity] [Priority: 3] {ICMP} 192.168.40.129 -> 192.168.40.128
11/29-11:26:53.695675  [**] [1:384:5] ICMP PING [**] [Classification: Misc activ
ity] [Priority: 3] {ICMP} 192.168.40.129 -> 192.168.40.128
11/29-11:26:53.695687  [**] [1:408:5] ICMP Echo Reply [**] [Classification: Misc
 activity] [Priority: 3] {ICMP} 192.168.40.128 -> 192.168.40.129
11/29-11:26:54.720073  [**] [1:366:7] ICMP PING *NIX [**] [Classification: Misc
activity] [Priority: 3] {ICMP} 192.168.40.129 -> 192.168.40.128
11/29-11:26:54.720073  [**] [1:384:5] ICMP PING [**] [Classification: Misc activ
ity] [Priority: 3] {ICMP} 192.168.40.129 -> 192.168.40.128
11/29-11:26:54.720098  [**] [1:408:5] ICMP Echo Reply [**] [Classification: Misc
 activity] [Priority: 3] {ICMP} 192.168.40.128 -> 192.168.40.129
11/29-11:27:00.650202  [**] [1:1917:6] SCAN UPnP service discover attempt [**] [
Classification: Detection of a Network Scan] [Priority: 3] {UDP} 192.168.40.1:53
761 -> 239.255.255.250:1900
11/29-11:27:01.660935  [**] [1:1917:6] SCAN UPnP service discover attempt [**] [
Classification: Detection of a Network Scan] [Priority: 3] {UDP} 192.168.40.1:53
761 -> 239.255.255.250:1900
11/29-11:27:02.671243  [**] [1:1917:6] SCAN UPnP service discover attempt [**] [
Classification: Detection of a Network Scan] [Priority: 3] {UDP} 192.168.40.1:53
761 -> 239.255.255.250:1900
11/29-11:27:03.671920  [**] [1:1917:6] SCAN UPnP service discover attempt [**] [
Classification: Detection of a Network Scan] [Priority: 3] {UDP} 192.168.40.1:53
761 -> 239.255.255.250:1900
```

Snort terminal resulting from the ping in Kali

```
761 -> 239.255.255.250:1900
11/29-11:27:46.724242  [**] [1:1421:11] SNMP AgentX/tcp request [**] [Classifica
tion: Attempted Information Leak] [Priority: 2] {TCP} 192.168.40.129:40941 -> 19
2.168.40.128:705
11/29-11:27:46.753132  [**] [1:1418:11] SNMP request tcp [**] [Classification: A
ttempted Information Leak] [Priority: 2] {TCP} 192.168.40.129:40941 -> 192.168.4
0.128:161
11/29-11:27:59.645095  [**] [1:1917:6] SCAN UPnP service discover attempt [**] [
Classification: Detection of a Network Scan] [Priority: 3] {UDP} 192.168.40.1:58
314 -> 239.255.255.250:1900
```

Snort's reaction to the nmap scan that was run on Kali



After opening port 80, legion was able to scan and determine the open port on the Ubuntu system.

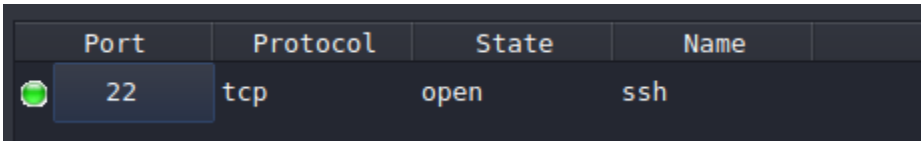
Answer the following questions:

1. What did you observe in each of the five times above you were asked to notice what happens?

Every time we were asked to check the snort terminal to determine what was happening, it would essentially warn the user that the system was being network scanned and provide some insight as to how.

2. Instead of opening port 80, on the Ubuntu machine, open port 22 and repeat the same steps we did in task 2 above. What protocol is shown in legion under services in this case? What did you notice in the Snort terminal when you did steps?

When I did task 2 and opened port 22, Legion found the ssh port to be open. The snort terminal was displaying warnings about nmaps. Legion's discovery is displayed below



Key Takeaways from Exercise 2:

- How to use Legion and configure it to run more advanced nmap scan/attacks on other machines
- How to identify what Snort was detecting on Ubuntu

Conclusion

In this lab we learned on intrusion detection systems worked on virtual machines by using Kali Linux and Ubuntu VMs. We started by installing and understanding Snort, then using it to detect and prevent intrusions on our Ubuntu machine.